

GFS

Sistema de Archivos de Google (GFS)

- Es un sistema de archivos distribuido propietario, que se creó como una solución para servir a las aplicaciones de transmisión de datos altamente concurrentes administradas por Google. Al igual que otros sistemas de archivos distribuidos.
- GFS tenía como objetivo proporcionar un sistema de almacenamiento confiable, escalable y de alta disponibilidad, que se basa en muchas de las mismas primitivas del sistema, desde el registro de operaciones hasta la replicación de datos, instantáneas, puntos de control y mucho más.

Sistema de Archivos de Google (GFS)

- *A pesar de muchas de estas similitudes con los sistemas existentes, el diseño de GFS está estrechamente acoplado con las cargas de trabajo de sus clientes, la adopción de la agregación de cantidades masivas de hardware básico combinado con un diseño tolerante a fallas y muchas otras decisiones arquitectónicas astutas le han permitido ser un sistema crítico en una de las principales compañías de datos del mundo.*

Supuestos de Diseños (GFS)

- **Read workload:** consiste principalmente en lecturas de grandes streaming (de varios cientos de KB a varios MB), con algunas lecturas aleatorias pequeñas.
- **Write workload:** consiste principalmente en grandes acoplamientos secuenciales a archivos (tamaño similar a las lecturas grandes), con pocas escrituras aleatorias pequeñas (los archivos tienden a no modificarse después de ser escritos).

Supuestos de Diseños (GFS)

- Archivos GFS: generalmente experimentarán **workload pesadas de acoplamientos altamente concurrentes**, y es crucial que el sistema implemente estas operaciones de manera eficiente.
- Los clientes tienden a participar en **grandes trabajos de procesamiento de datos**, que dependen de mantener un ancho de banda alto y sostenido, más que de optimizar la latencia de operación individual

GFS clusters

- GFS Cluster
 - A master
 - Multiples chunkservers
 - Acceso concurrente por varios clientes

Master

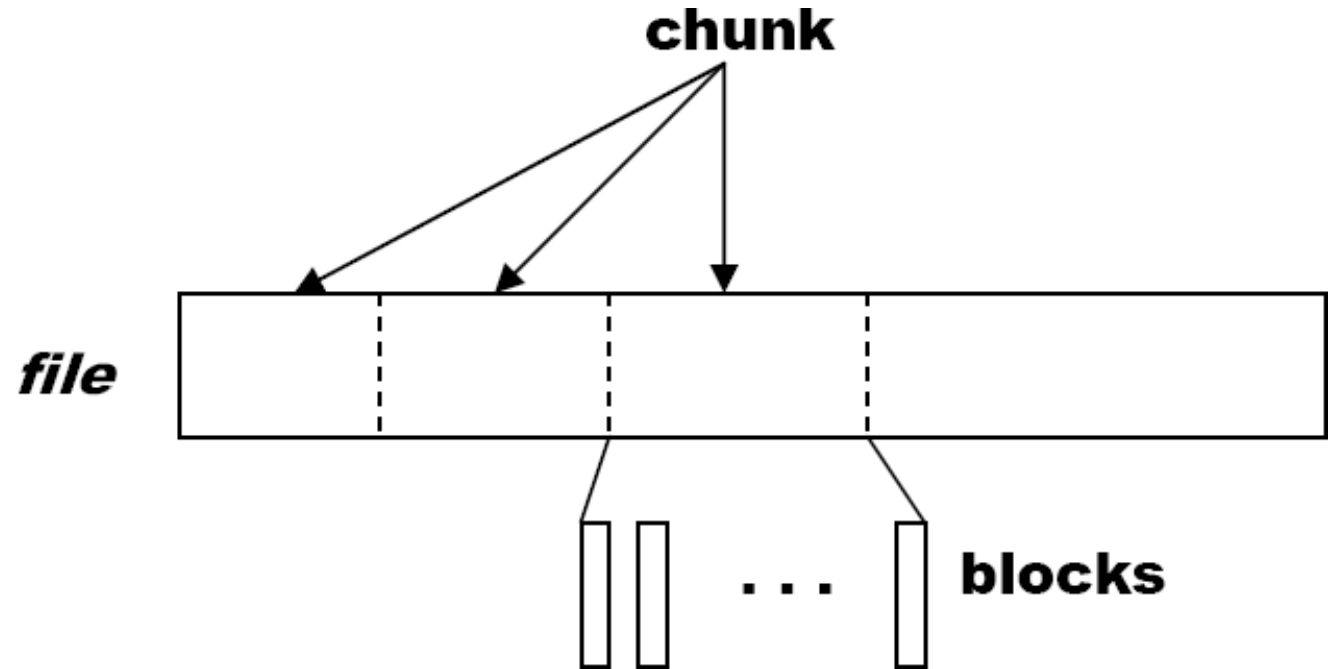
Chunkserver

Chunkserver

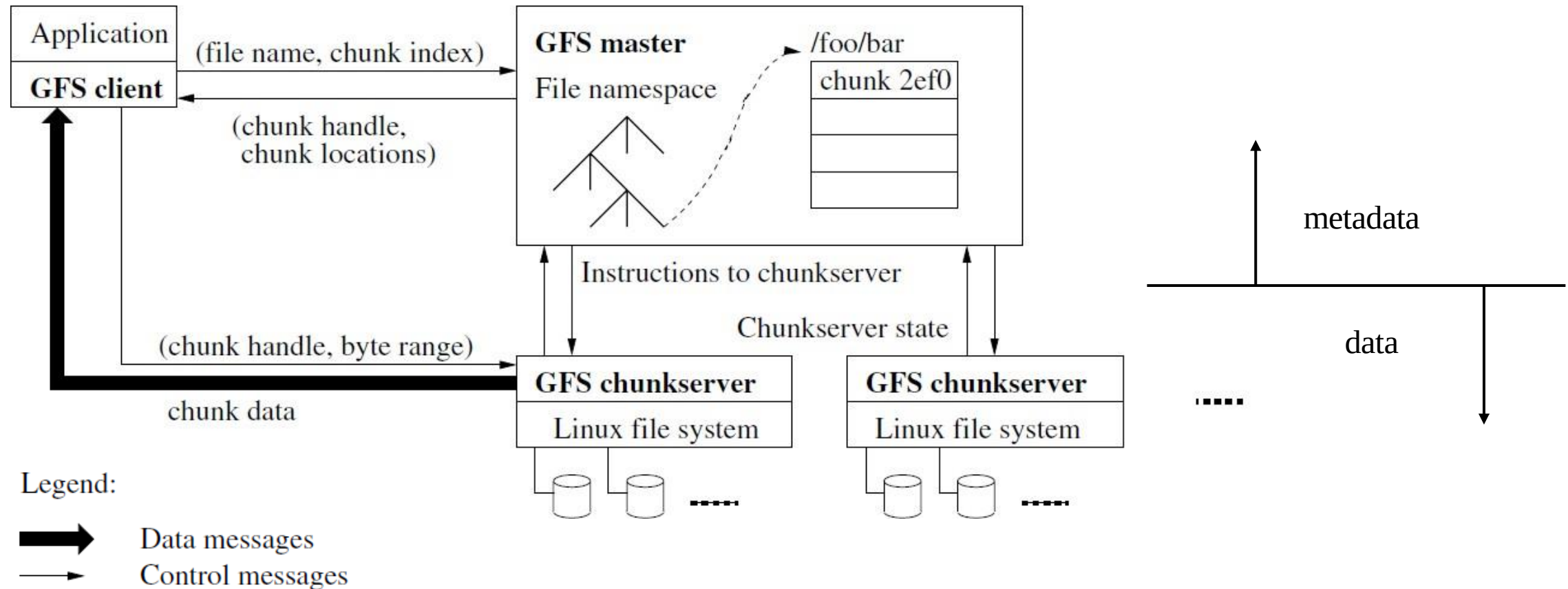
Chunkserver

Estructura de archivos

- Dividido en chunks de 64 MB
- Chunk identificado por un identificador de 64 bits
- Chunks replicados (3 replicas por defecto)
- Chunks dividido en boques (blocks) de 64KB
- Cada block tiene un checksum de 32-bit



High-level Architecture



High-level Architecture

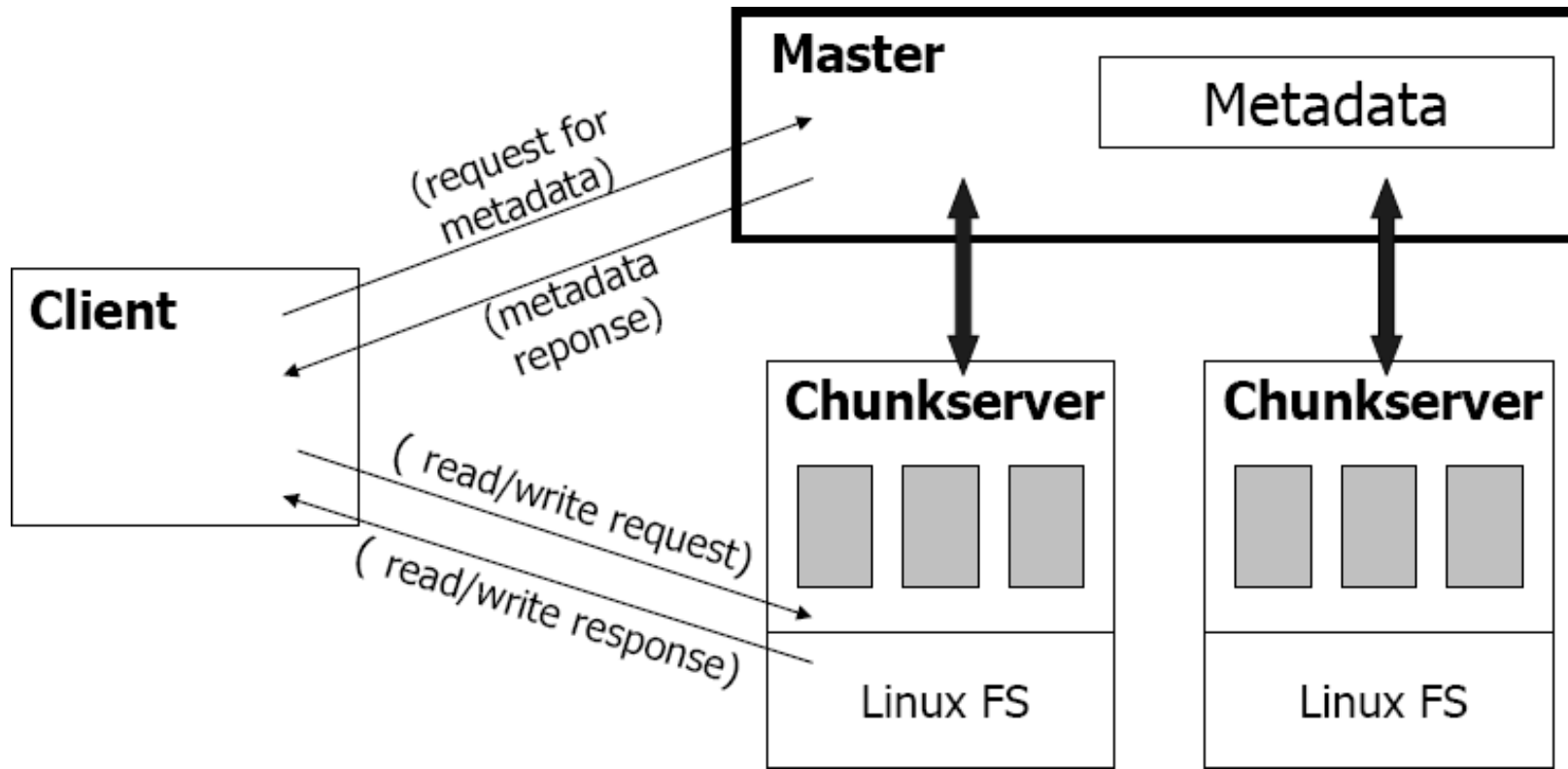
Master

- Administra namespace/metadata
- Administra creación, replicación y asignación del chunk
- Realiza una operación de creación de instantaneas del arbol de archivos para crear duplicado.
- Realiza puntos de control y registra cambios en los metadatos de los servidores.

Chunkservers

- Almacena chunks y realiza sumas de verificación para cada bloque.
- Reporta al master procesos de arranque/recuperación de fallos
- Informa periódicamente al servidor master sobre sub conjuntos de fragmentos que ya no son necesarios.

High-level Architecture



High-level Architecture

- Los clústeres GFS constan de un único nodo maestro, que almacena todos los metadatos del sistema de archivos, incluidos los siguientes:
 - Espacio de nombres de file y chunk
 - Mapeo de file-to-chunk
 - Locaciones de replicas de chunks

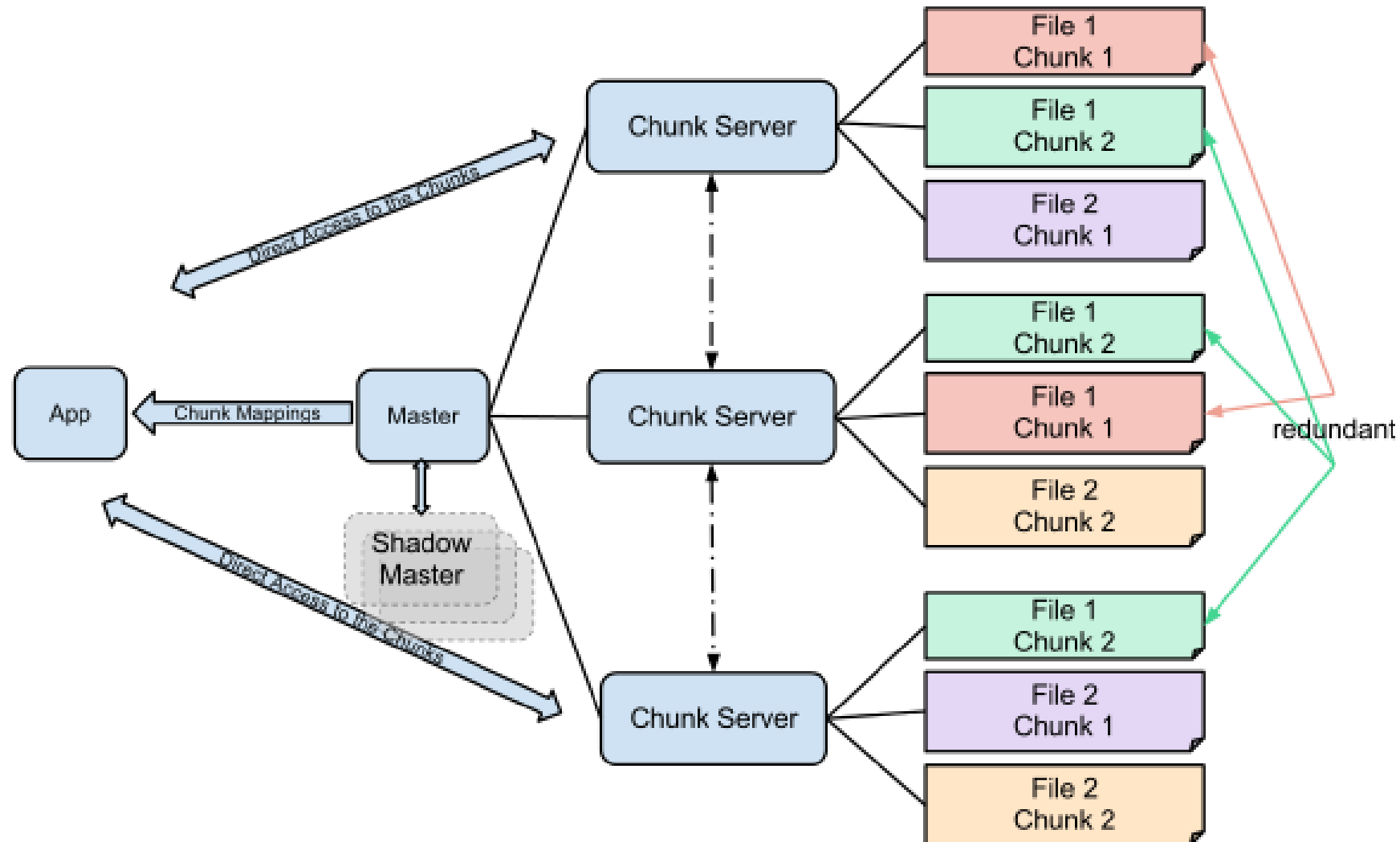
High-level Architecture

- *Una de las decisiones de diseño tomadas para optimizar enormemente GFS es la combinación de archivos grandes y pocos metadatos almacenados por **chunk**. El tamaño de los metadatos por **chunk** es de aproximadamente 64 bytes, y dado que los archivos almacenados en GFS suelen ser muy grandes, el número total de archivos que deben tenerse en cuenta es bastante bajo. Por esta razón, todos los metadatos GFS se almacenan en memoria, y esto optimiza en gran medida las operaciones de metadatos en su conjunto.*

High-level Architecture

- Además del nodo maestro, los clústeres GFS consisten en un número variable de nodos **chunkserver**, a través de los cuales los datos se particionan y replican. Los **Chunks** se almacenan en nodos de **chunkserver** (almacenados como archivos linux) y se accede a ellos mediante sus respectivos **chunks handles** combinados con un byte offset.
- Dado que las ubicaciones de réplica de **chunks** se pueden determinar comunicándose con todos los **chunks servers** en el inicio, estos son los únicos metadatos no persistente por el **master**.

High-level Architecture



Arquitectura GFS

Desacoplamiento del flujo de datos y el control

- La separación de los metadatos del sistema de archivos (almacenados únicamente por el master) y los datos de archivo (almacenados en **chunks servers**) garantiza que el único master no resulte en un cuello de botella.
- *"El cliente traduce el nombre del archivo y el byte offset... en un **chuck index**... esto envía al master una solicitud que contiene el nombre de archivo y el **chuck index**. El master responde con el identificador de **chuck handle** y **location** de las réplicas. El client almacena en caché esta información..."*

Arquitectura GFS

- A partir de este momento, cualquier operación de datos no requiere interacción entre el **client** y el nodo **master** y se puede enviar directamente al **chunk replica**, mientras que el maestro continúa sirviendo operaciones de metadatos.
- La importancia de que el **master** solo esté involucrado en las operaciones de metadatos viene en la simplicidad. Esto simplifica la forma en que se manejan el **namespace** y los metadatos, ya que solo hay una única fuente de verdad, y también simplifica las decisiones que deben tomarse para la colocación y replicación de **chunk**.

Log de operaciones (operation log)

- El Operation Log tiene un propósito similar al de un registro de transacciones de base de datos y se utiliza para garantizar una timeline lógica coherente de las mutaciones de metadatos del sistema de archivos. En cualquier mutación de metadatos, el master primero registra estas operaciones en un orden serial en el registro de operaciones y replica el registro, antes de confirmar las mutaciones y responder al cliente.

Operation Log

- *El mantenimiento de un **operation log** garantiza que podamos restaurar el estado del **master** simplemente reproduciendo la serie de mutaciones escritas en el **log**, así como asegurando que las operaciones fallidas no dejen los metadatos del sistema de archivos en un estado dañado.*
- Para optimizar aún más la restauración del estado **master**, GFS toma **checkpoints**, que abarcan una serie de operaciones en el **operation log** , lo que permite que el registro se trunque solo a las operaciones post-checkpoints, que se pueden reproducir después de aplicar el checkpoints para restaurar el estado del master. Estos checkpoints son persistentes en el disco, con un puñado de **operation log** anteriores y **checkpoints** también persistentes para la recuperación ante desastres.

Chunk Leases (arrendamiento de chunks)

- Dado que el master no está involucrado en las operaciones de datos, debe transferir la potencia a una de las chunk replicas para determinar un orden de mutación serializado. Esto se administra asignando un **lease (bandera)** a una de las réplicas para un **chunk** en particular.
- Los Chunk Leases se revocan después de un breve timeout, o pueden ser revocados manualmente por el **master** si el **chunk** necesita ser protegido de cualquier mutación. Sin embargo, en su mayor parte, los **chunk** que están mutando constantemente no tendrán sus **leases** revocados, ya que los **chunkservers** pueden solicitar extender estos **leases** a través de la comunicación con el **master**.

Chunk Leases

- Los client consultarán inicialmente el master para ver que réplica de chunk contiene el lease y la locations de todas las réplicas (para el chunk que están mutando). Esta información será devuelta por el master y almacenada en caché para su uso continuo (el maestro seleccionará una réplica de chunk para otorgarle lease si ninguna ya tiene una).
- A partir de este momento, el client puede simplemente enviar los datos para la operación, y estas operaciones se propagarán a todas las réplicas. Finalmente, una vez que todas las réplicas hayan reconocido que han recibido los datos, el client emitirá la **write request** a la réplica primaria (la réplica que posee el lease), que determinará un orden en serie para las mutaciones, que posteriormente se aplicará de la misma manera a todas las réplicas.

Chunk Leases

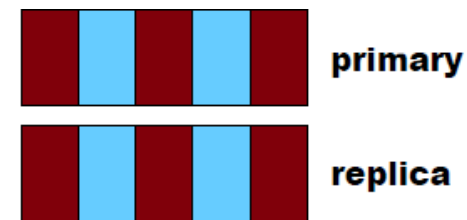
- Dado que la transmisión real de datos del cliente a las réplicas se desacopla de la determinación del orden dado por el primario y la posterior commit de las mutaciones, esto también crea un elemento de optimización de la red, donde el flujo de datos puede basarse en la topología de red de las réplicas de chunk, en lugar de enviar primero los datos al chunk primario.

Consistency Model

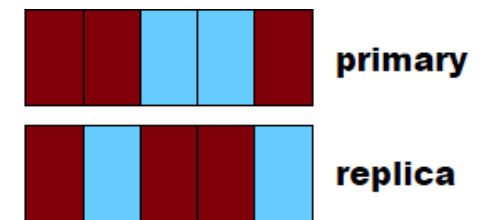
- El Consistency Model de GFS es consistente con la filosofía general de GFS, diseñado para ser flexible a la workload típica de los clientes. Se basa en la idea de que las mutaciones siguientes, que pueden o no tener éxito, y tal vez concurrentes, regiones en un archivo pueden estar en uno de los tres estados.
 - consistent
 - defined
 - undefined



defined



consistent



inconsistent

Consistency Model

- consistent: los clientes ven los mismos datos independientemente de la réplica leída, las mutaciones concurrentes pueden dejar los datos consistentes, pero los datos pueden estar formados por fragmentos de múltiples mutaciones
- defined: la región es consistente y la mutación tiene éxito sin interferencia de escritores concurrentes (todos los clientes ven lo que la mutación ha escrito)
- undefined: diferentes clientes pueden ver datos diferentes (resultado de una mutación fallida)

Consistency Model

- *Los clientes de GFS solo necesitan poder distinguir entre regiones definidas e indefinidas.*
- *Esto se debe a la naturaleza del orden de mutación en serie seleccionado por la réplica de chunk que actualmente tiene el lease, lo que significa que las mutaciones se aplicarán de la misma manera a todas las réplicas.*
- *Además, dado que GFS utiliza el control de versiones para los chunks, los chunk obsoletos no tendrán mutaciones aplicadas y el master no enviará sus ubicaciones al cliente, lo que garantiza que todas las réplicas que se puedan leer sean coherentes entre sí.*

Consistency Model

- Aunque las réplicas de chunk obsoletos pueden tener sus ubicaciones almacenadas en caché por el cliente (y, por lo tanto, se pueden leer desde el), la carga de trabajo de anexo pesado de los clientes GFS tiende a minimizar los efectos adversos de la lectura de datos obsoletos.
- Es importante reconocer que las razones por las que GFS es capaz de relajar su modelo de consistencia se deben a una comprensión completa de los workloads de sus clientes. En particular, GFS enfatiza que los clientes confían en la adición para la mayoría de las escrituras, checkpoints y escritura de registros de autovalidación, como medio para garantizar que el modelo de consistencia relajada no afecte negativamente al cliente.

Mutación de datos(Data mutations)

Writes:

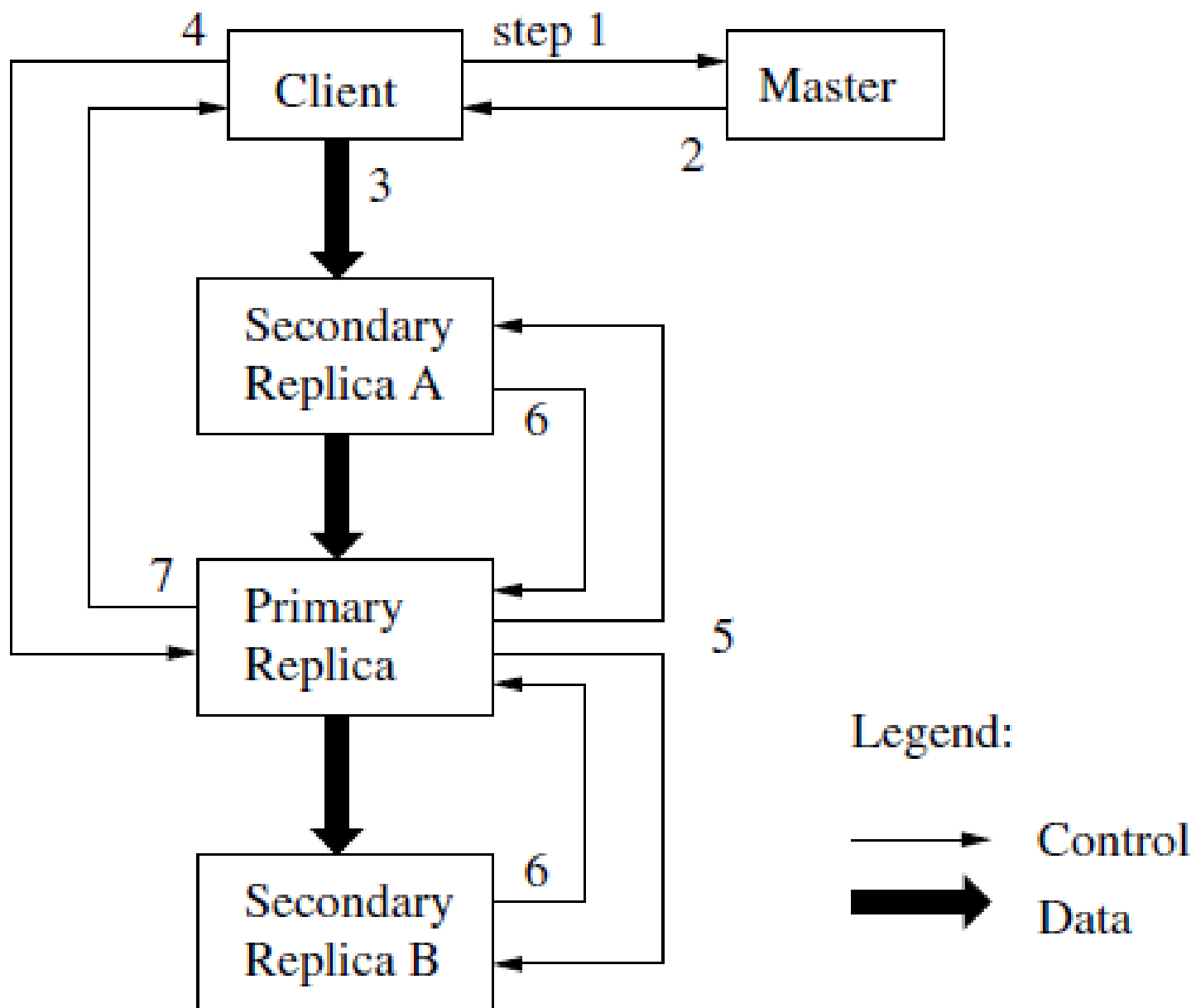
- Provocar que los datos se escriban en un desplazamiento específico (offset).

Record appends:

- Provocar que los datos se agreguen automáticamente al menos una vez en un desplazamiento de GFS elegido

Consistency

- Aplicar mutaciones a un fragmento en el mismo orden
- Utilizando números de versión de fragmento



Mutations

- 1. El cliente solicita un arrendamiento al servidor maestro.
- 2. El servidor maestro otorga permiso de actualización a un cliente durante un período de tiempo limitado (60 segundos).
- 3. El cliente envía los datos a todas las réplicas.
- Los datos finalizan en la caché de búfer LRU interna de cada servidor chunk.
- 4. Una vez que todas las réplicas hayan confirmado la recepción de los datos, el cliente envía una solicitud de escritura a la réplica principal.
 - Primary asigna un número de serie a la mutación y lo aplica a su estado local.

Mutations

- 5. Las réplicas primarias reenvían la solicitud de escritura a todas las réplicas secundarias, que aplican la mutación en el mismo orden secuencial.
- 6. Las réplicas secundarias responden a la principal una vez que han completado la operación.
- 7. El sistema principal notifica al cliente que la mutación se ha completado.

Atomic record appends

- GFS agrega los nuevos datos
 - Al menos una vez
 - De forma atómica
 - En un desplazamiento elegido por GFS
- Devuelve ese desplazamiento al cliente
- Ampliamente utilizado para implementar el acceso concurrente

Snapshots

- Copia archivos y directorios en paralelo con las operaciones habituales.
- Utiliza el método de copia en escritura.
- Hace que los datos copiados sean temporalmente de solo lectura.
- Para detectar los cambios que se produzcan mientras se toma la instantánea.

Arquitectura GFS

Replica Placement

- La Replica Placement de chunk se produce por varias razones, que son las siguientes:
 - **replicating chunks** en la creación para proporcionar integridad de datos
 - **re-replicating chunks** cuando las réplicas existentes se han quedado obsoletas o los servidores de fragmentos dejan de responder
 - **load balancing disk** utilización a lo largo de los **chunkservers**

Arquitectura GFS

- Los factores considerados al colocar réplicas de chunk son la elección de chunkservers con baja utilización de discos (para utilizar uniformemente los discos en los chunkservers), evitar la creación de muchos chunks en un solo chunkserver a la vez y distribuir réplicas entre bastidores. El primer punto se explica por sí mismo y sirve como una heurística de equilibrio de carga. El segundo punto se debe a los patrones de workload de los clientes GFS, que normalmente transmiten grandes cantidades de escrituras a chunks recién creados (evitando así los puntos calientes que se producirían si se colocan muchas réplicas al mismo tiempo en el mismo chunkserver). El último punto es una capa adicional de tolerancia a fallos que se agrega a través de la propia colocación de réplicas.

Arquitectura GFS

- Los chunk son re-replicated cuando el recuento de réplicas en buen estado cae por debajo del umbral configurado, que por defecto es 3, pero se puede configurar dinámicamente.
- Finalmente, el master equilibrará periódicamente la carga de réplicas de chunk como una tarea en segundo plano. Este equilibrio de carga sigue la misma heurística de la colocación inicial de la réplica, pero también tiene un elemento adicional de intentar eliminar una réplica en un chunkserver con una utilización de disco superior a la media, en comparación con una de un chunkserver de utilización de disco inferior a la media.

Garbage Collection

- Al igual que la asignación de archivos GFS, que se ejecuta lazily, el **garbage collection** no utilizados es una tarea en segundo plano manejada lazily, que es una combinación de comunicación periódica entre los **chunkservers** y el **master**, y el escaneo y la modificación de los **file system metadata** por parte del **master**.
- Cuando los archivos se eliminan, simplemente se “**hidden**” por su nombre solo en el **file system namespace**, y los chunks no se recuperan de inmediato (lo que permite una ventana de tiempo en la que pueden leerse por su nombre hidden e incluso no eliminarse).

Garbage Collection

- Después de algún tiempo, el **master** eliminará el archivo, lo eliminará del **namespace**, lo que purgará todos sus metadatos **in-memory**, separando todas las asignaciones del archivo eliminado a los **chunk** en los que consistía. El master eventualmente buscará todos los **chunks** huérfanos y también eliminará sus metadatos.
- Dado que hay una asignación clara de archivos a **chunk** que se mantiene completamente por un solo **master**, el garbage collection es especialmente fácil para GFS, y es simplemente una cuestión de identificar fragmentos **chunk**, purgar sus metadatos e informar a los chunkservers.

Garbage Collection

- Periódicamente, los chunkservers y el master intercambiarán mensajes, donde el chunkserver enviará las réplicas de chunk que mantiene al master, mientras que el master responde con los chunks que ya no están presentes en los metadatos, que los chunkservers son libres de reclamar.
- Este proceso asincrónico de purgar primero los metadatos de los chunks y, finalmente, recuperarlos a través de la comunicación es muy simple, y también se presta a ser útil cuando la creación de chunk falla y deja chunks que no son utilizables, que luego se recuperarán a través de la comunicación entre el chunkservers y el master.

Garbage Collection

- Este proceso de garbage collection también permite la amortización de la recuperación del almacenamiento, al agrupar (batching) chunks para recuperar para cada chunkserver, en lugar de la avaricia uno a la vez.

Key Takeaways

- simplificación de las operaciones de metadatos y la colocación/administración de réplicas con una única arquitectura maestra.
- desacoplamiento de las operaciones de metadatos de las operaciones de datos y uso de chunk leases, para evitar involucrar al master en la serialización del orden de mutación concurrente.
- confiar en los puntos de control y el registro para obtener coherencia y una recuperación rápida del master.
- implementar métodos sólidos para identificar y remediar fallas en los componentes

Key Takeaways

- asignación de discos lazy y garbage collection través de mensajería y purga de metadatos.
- separar el flujo de datos de la commit de mutaciones en chunk.
- diseñar agresivamente todos los elementos del sistema para el workload del client.